Creá tu Proyecto con Inteligencia Artificial



Cómo se construyen las aplicaciones de verdad

CLASE 04

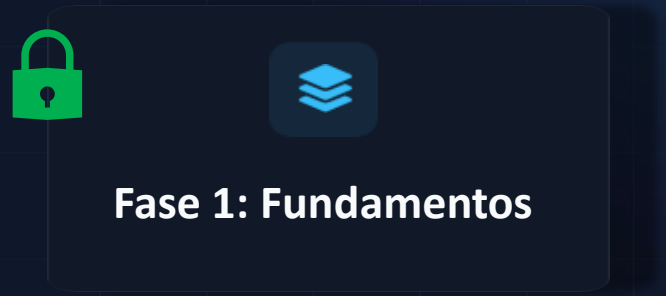
CENTRO DE GRADUADOS DE LA FACULTAD DE INGENIERÍA

TU EXITO PROFESIONAL

Del prototipo en el navegador a la arquitectura desacoplada

Hoy cerramos formalmente la Fase 1 de diseño y auditoría para dar el gran salto de madurez técnica a la Fase 2: Arquitectura y Software Real.

Hasta la semana pasada trabajamos con especificaciones formales, matrices de roles y prototipos interactivos en un único archivo dentro del navegador.





Hoy pasamos del prototipo a un sistema real creando nuestro primer backend (Python, FastAPI y base de datos relacional) para persistir los datos y la lógica de negocio.

La Evolución de Nuestro Sistema de Turnos

1. spec.md

(Clase 2 y 3): El Contrato. Reglas invariantes

2. plan.md

(Clase 3): La Estrategia. Cómo desacoplar la interfaz de la lógica central.

3. tareas.md

(Clase 3): El Checklist de ejecución atómica paso a paso.

4. index.html

(Clase 1 a 3): La interfaz web del turnero operando con datos temporales en memoria local.

Miren esta línea de tiempo. Lo que construimos no se descarta; se acumula.

DIAGNÓSTICO TÉCNICO

El Límite de la “MAQUETA”

¿Por qué un solo archivo index.html no puede ir a producción?

MEMORIA VOLÁTIL

Al recargar el navegador, los datos desaparecen porque no existe una base de datos.



INSEGURIDAD ESTRUCTURAL

Las reglas de negocio y los permisos (RBAC) viven en el navegador; cualquier usuario puede manipular el JavaScript localmente.



CERO CONCURRENCIA

Dos personas abriendo la web al mismo tiempo no ven los cambios del otro porque no existe un punto central de sincronización.



Taller de Arranque: Git Clone

Vamos todos a la acción. Para que nadie se quede atrás si tuvo alguna dificultad en la clase pasada, hoy arrancamos todos exactamente desde la misma línea de largada.

Via terminal:

```
git clone https://github.com/centrograduadosFIUBA/turnos-prototipo-conceptual.
```

Via Visual Studio Code:

Abrir VS Code en blanco.

Menú de comandos (Ctrl+Shift+P / Cmd+Shift+P) -> Git: Clonar

Pegar el enlace del repositorio del turnero.

Seleccionar carpeta local y hacer clic en "Abrir repositorio"

Anatomía de una App

1. Frontend (La Interfaz)

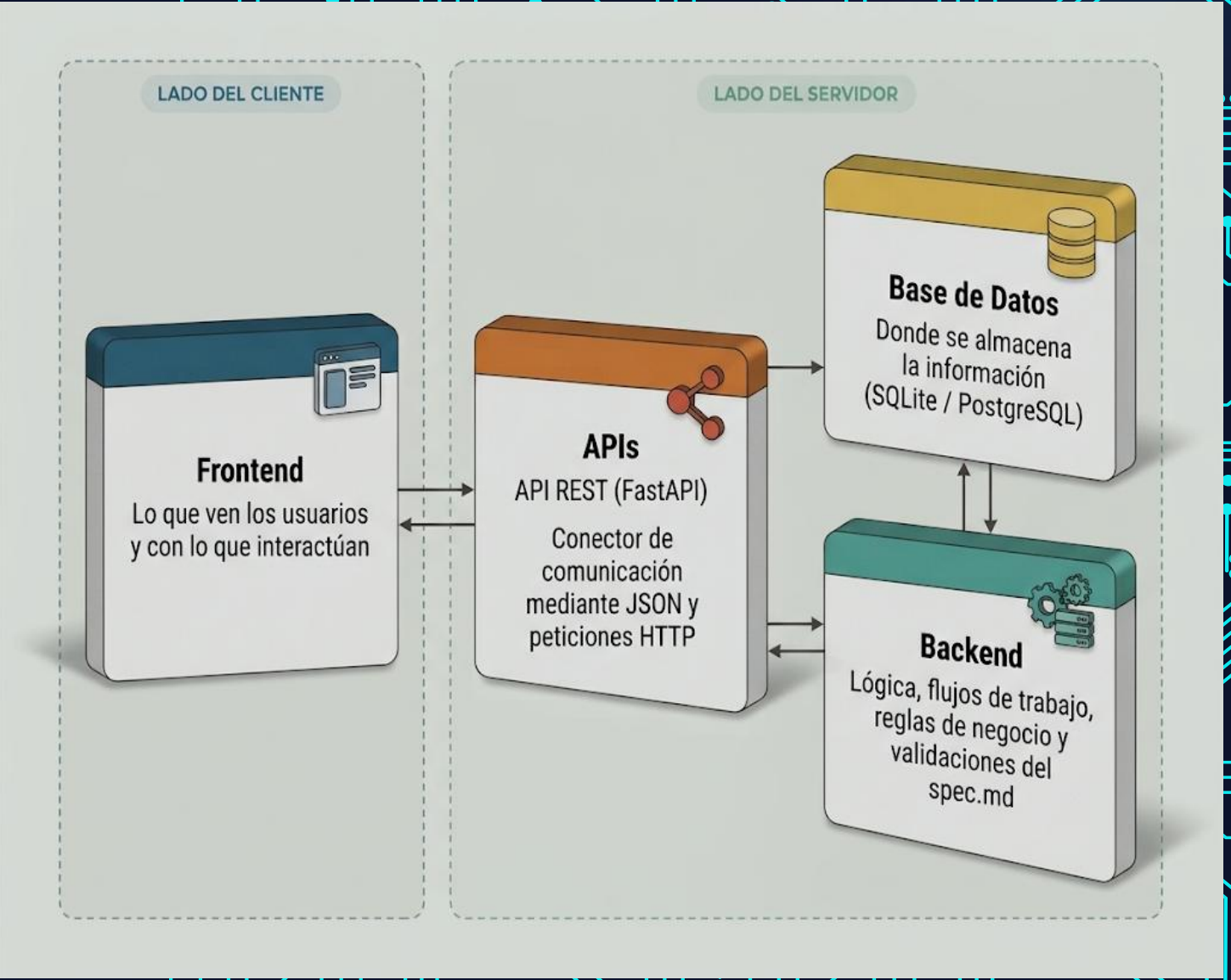
La capa visual con la que interactúa el usuario (pantallas, botones, formularios y tablas).

2. Backend (El Motor)

El servidor que procesa la lógica, guarda datos y gestiona la seguridad del sistema.

3. API REST (Conector)

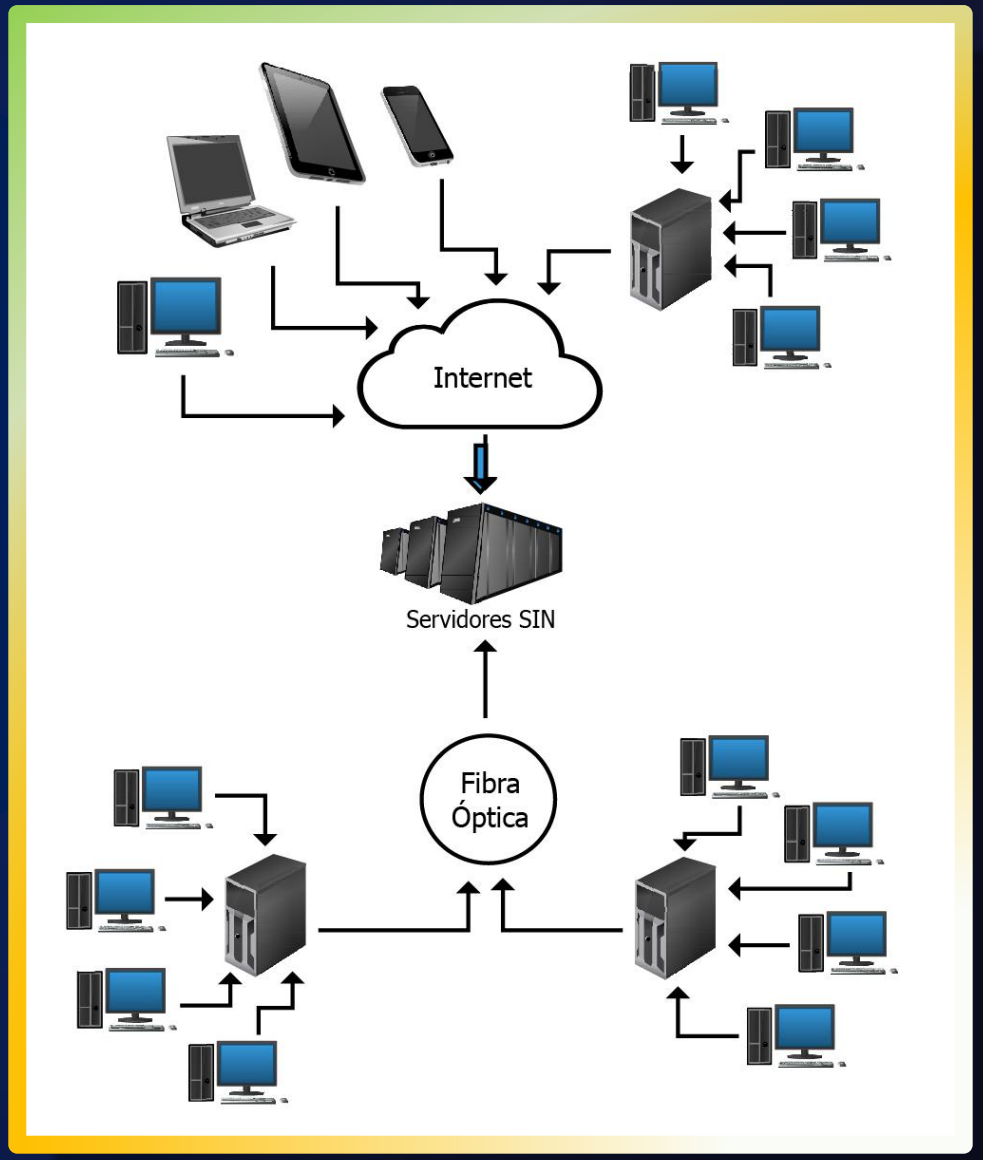
Maneja la comunicación entre el frontend y el backend.



INTERNET

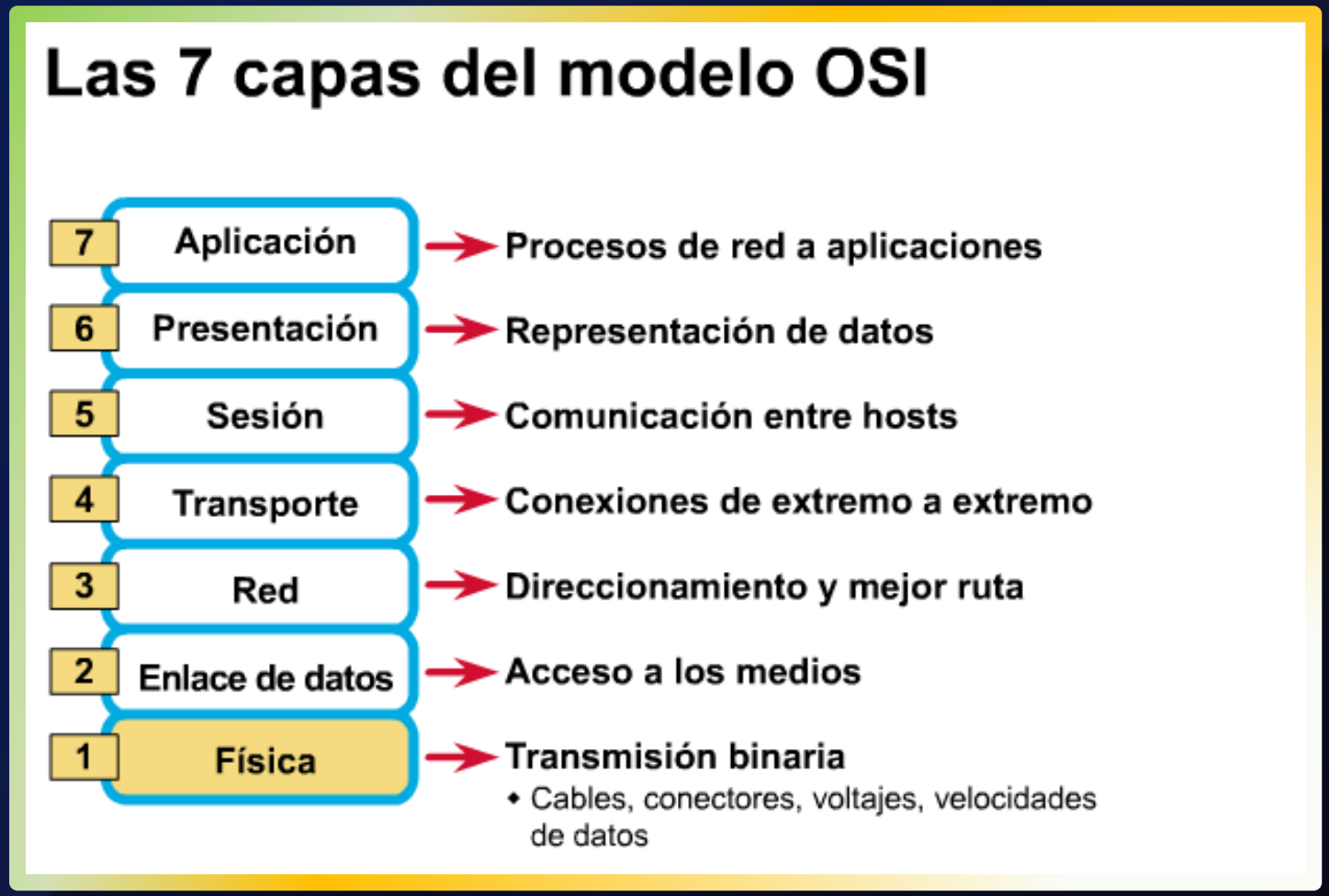
Internet es mucho mas que computadoras y servidores conectados entre si.

Para poder mandar información de un puto a otro, debe existir cierta organización, protocolos de comunicación y estándares adoptados por todos.



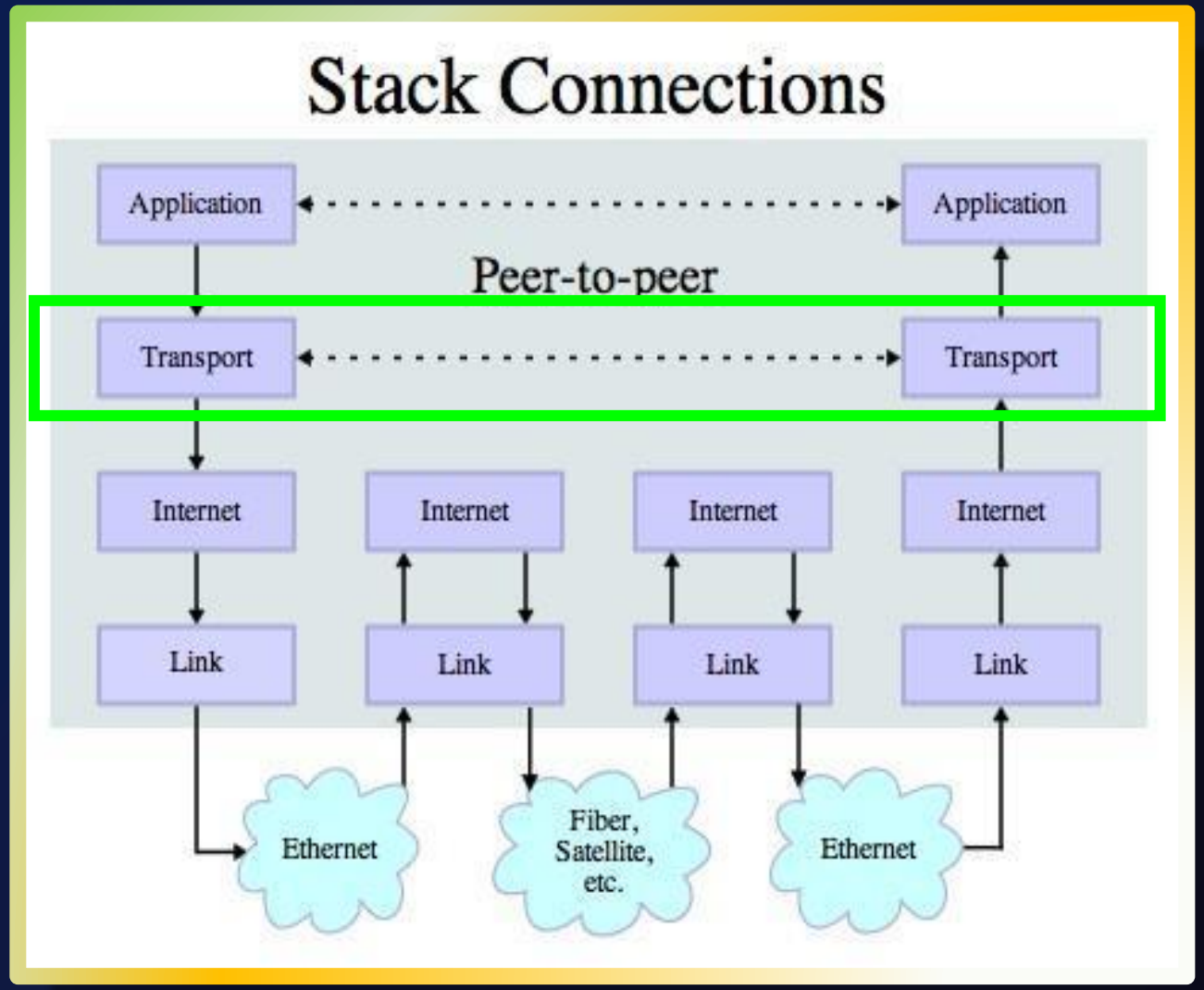
INTERNET las capas

El modelo OSI, o Open Systems Interconnection, es un marco conceptual desarrollado por la Organización Internacional de Normalización (ISO) en la década de 1980. Su propósito es facilitar la interconexión y comunicación entre diferentes sistemas de red mediante la estandarización de protocolos.



TRANSPORT CONTROL PROTOCOL (TCP)

- Construido sobre IP (Internet Protocol)
- Asume que IP podría perder algunos datos: almacena y retransmite datos si parece que se han perdido.
- Maneja el "control de flujo" usando una ventana de transmisión.
- Proporciona una tubería (pipe) agradable y confiable



Fuente: http://en.wikipedia.org/wiki/Internet_Protocol_Suite

CONEXIONES TCP / SOCKETS

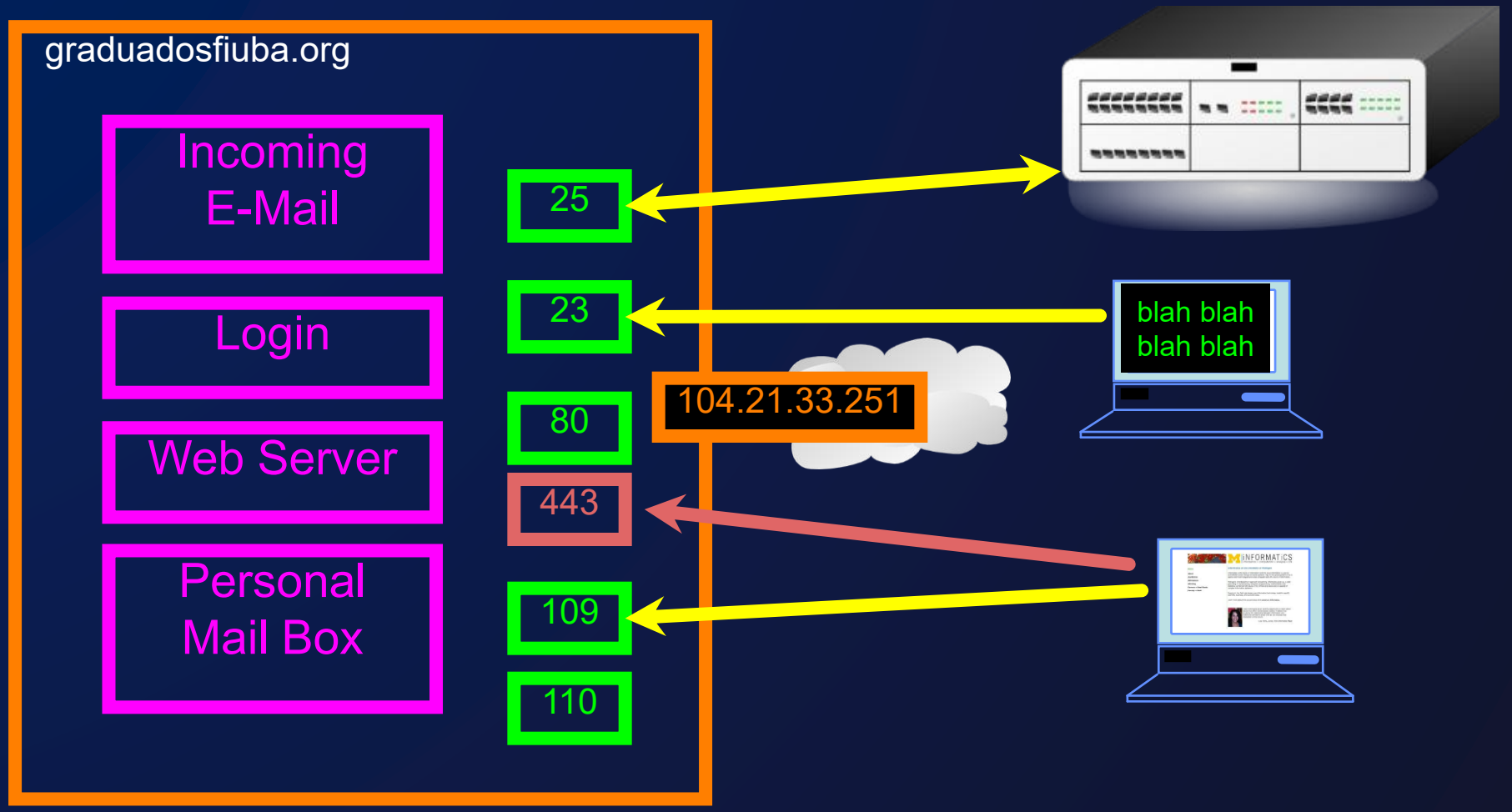
En redes informáticas, un socket de Internet o socket de red es un punto final de un flujo de comunicación bidireccional entre procesos a través de una red informática basada en el Protocolo de Internet.



Puertos TCP

- Un puerto es un punto final de comunicaciones de software específico de la aplicación o del proceso.
- Permite que varias aplicaciones en red coexistan en el mismo servidor.
- Hay una lista de números de puerto TCP conocidos.

PUERTOS TCP


graduadosfiuba.org
Incoming E-Mail
Login
Web Server
Personal Mail Box
25
23
80
443
109
110
104.21.33.251
blah blah
blah blah

PROTOCOLO DE APPLICATION

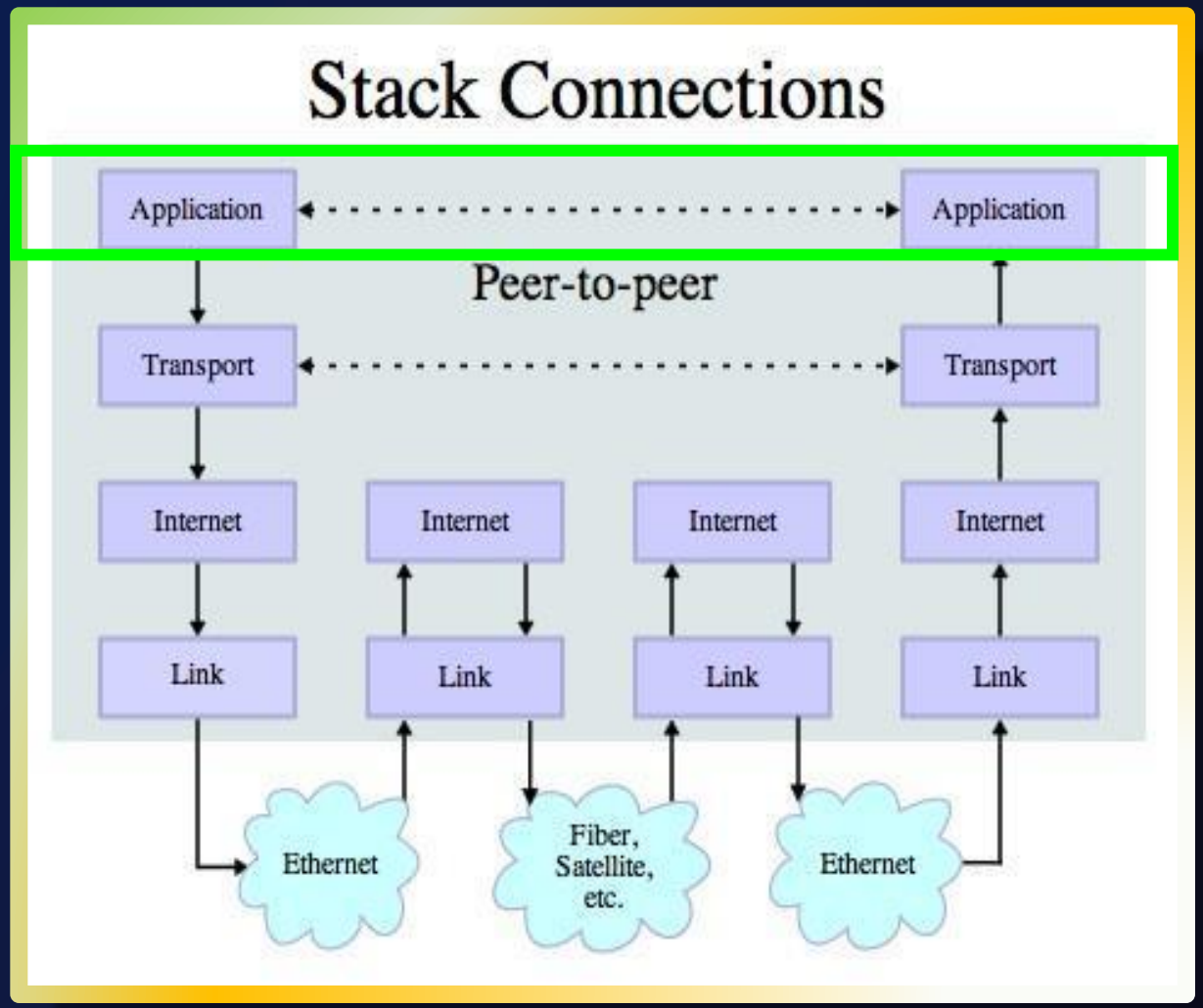
El protocolo de capa de aplicación dominante en Internet Inventado para la Web - para recuperar HTML, imágenes, documentos, etc.
Es HTTP - Hypertext Transfer Protocol

Es el conjunto de reglas que permiten a los exploradores recuperar documentos Web de servidores a través de Internet.

URL: Uniform Resource Locator

`https://graduadosfiuba.org/page1.html`

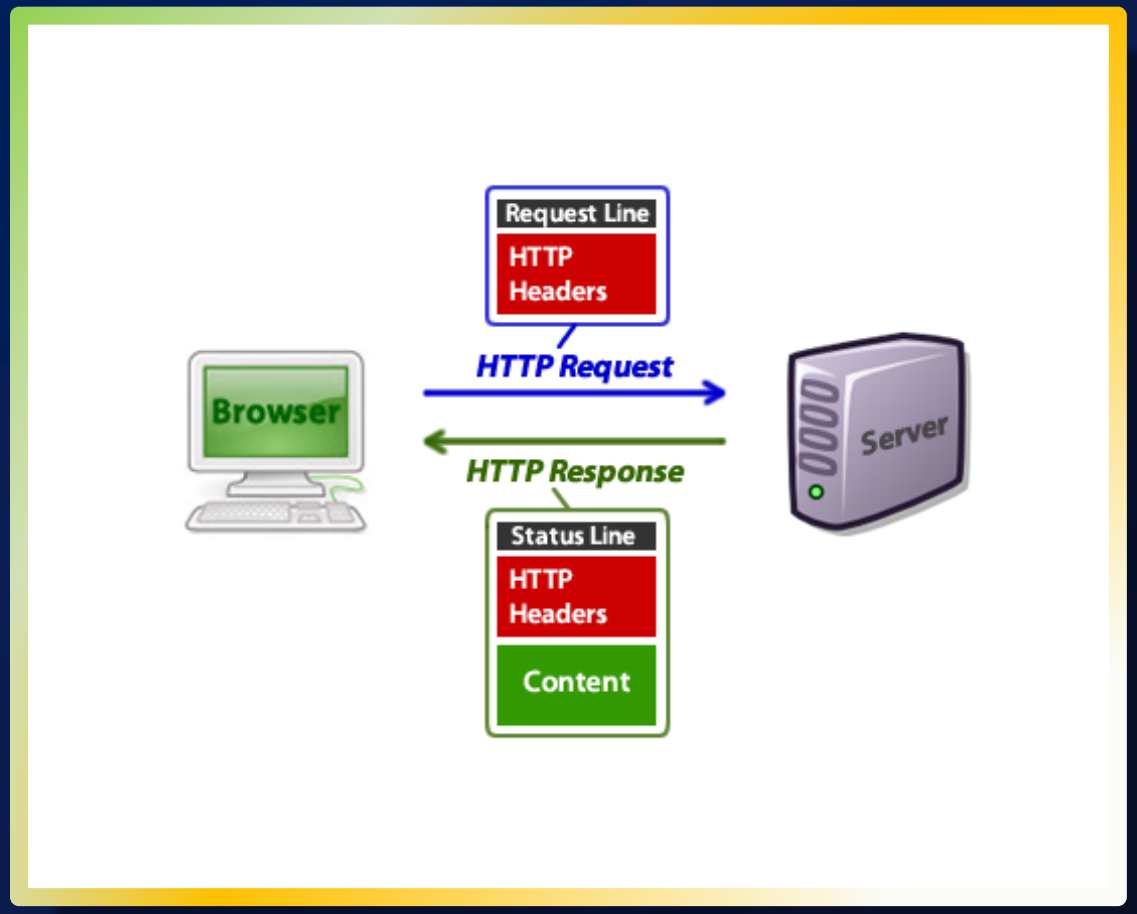
protocolo host documento



Fuente: http://en.wikipedia.org/wiki/Internet_Protocol_Suite

PETICIONES

Existen varios tipos de peticiones en HTTP, cada una diseñada para realizar diferentes acciones en el servidor. Los métodos más comunes son:



GET: Este método solicita la representación de un recurso específico del servidor. Es el más **utilizado para recuperar datos** sin modificar el estado del recurso. Por ejemplo, al acceder a una página web, se envía una petición GET para obtener su contenido.

POST: Se **utiliza para enviar datos** al servidor, como cuando se envían formularios o se suben archivos. A menudo, este método provoca cambios en el estado del servidor, como la creación de nuevos registros.

PETICIONES

PUT: Este método **reemplaza todas las representaciones actuales** del recurso de destino con la carga útil de la petición. Se usa comúnmente para actualizar recursos existentes.

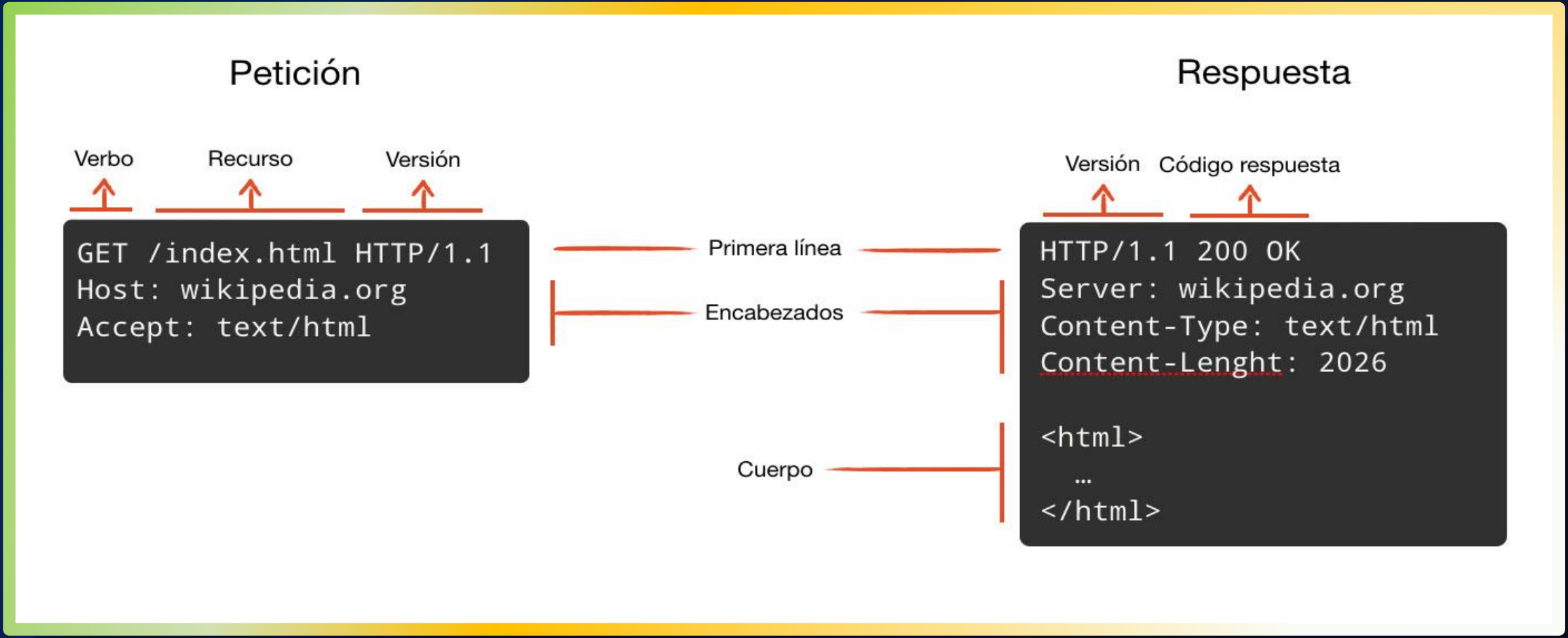
DELETE: Como su nombre indica, este método se utiliza para **eliminar un recurso específico** del servidor

PATCH: Se utiliza para **aplicar cambios parciales a un recurso existente**, en lugar de reemplazarlo por completo como lo hace PUT.



Existen otros métodos para realizar tareas con el servidor como HEAD, OPTIONS, CONNECT, TRACE

PETICIONES Y RESPUESTAS

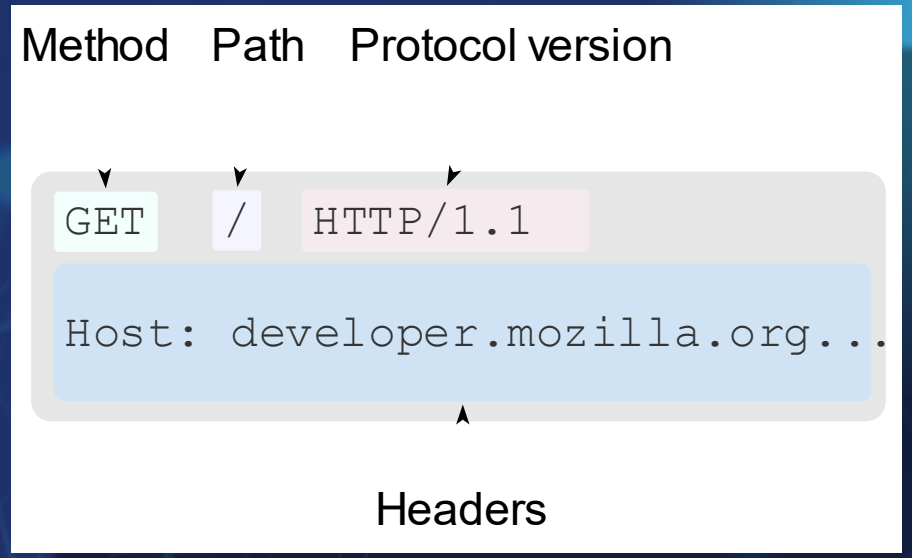


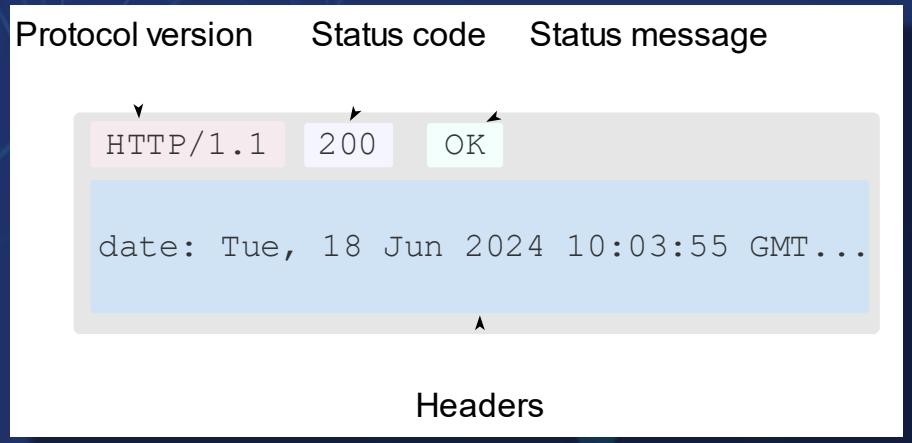
HTTP es el protocolo que permite que los navegadores pidan contenido (requets) a los servidores web y reciban una respuesta (response) que pueden mostrar al usuario.

Métodos comunes en HTTP

- **GET:** solicita datos (por ejemplo, ver una página).
- **POST:** envía datos (por ejemplo, al hacer login o subir un formulario).
- **PUT:** actualiza un recurso.
- **DELETE:** elimina un recurso.

Estas acciones son la base de cualquier **API REST**, que es un sistema que expone sus recursos a través de **URLs** y te permite manipularlos (CRUD) usando los métodos HTTP estándar.





Mas info sobre HTTP: https://developer.mozilla.org/es/docs/Web/HTTP

CENTRO DE GRADUADOS DE LA FACULTAD DE INGENIERÍA
TU EXITO PROFESIONAL

Respuestas – Status Code en HTTP

Los códigos de estado se agrupan en cinco categorías para que puedas saber rápidamente el tipo de respuesta:

- **1xx (Respuestas informativas):** La petición fue recibida y el proceso continúa.
- **2xx (Respuestas satisfactorias):** La petición fue recibida, entendida y aceptada con éxito.

  `200 OK: La solicitud ha tenido éxito.`
- **3xx (Redirecciones):** Se necesita tomar una acción adicional para completar la petición.

  `301 Moved Permanently: El recurso ha sido movido permanentemente a una nueva URL`

Respuestas – Status Code en HTTP

- **4xx (Errores del cliente):** La petición contiene una sintaxis incorrecta o no puede ser procesada.

  `400 Bad Request: El servidor no pudo interpretar la solicitud.`

  `403 Forbidden: No tienes permiso para acceder a este recurso.`

  `404 Not Found: El servidor no pudo encontrar el recurso solicitado.`

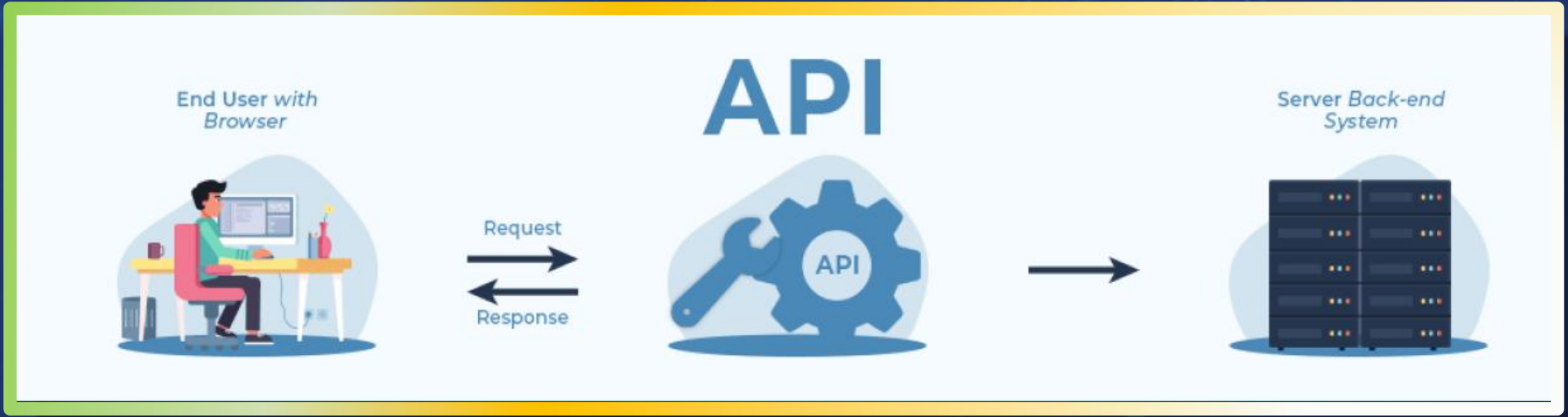
- **5xx (Errores del servidor):** El servidor falló al intentar cumplir con una petición aparentemente válida.

  `500 Internal Server Error: Un error genérico del servidor.`

En resumen, el ciclo de petición y respuesta HTTP es la base fundamental de cómo funciona la World Wide Web, permitiendo una comunicación estructurada y predecible entre tu dispositivo y los servidores de todo el mundo.

Protocolo de Transferencia de Hipertexto

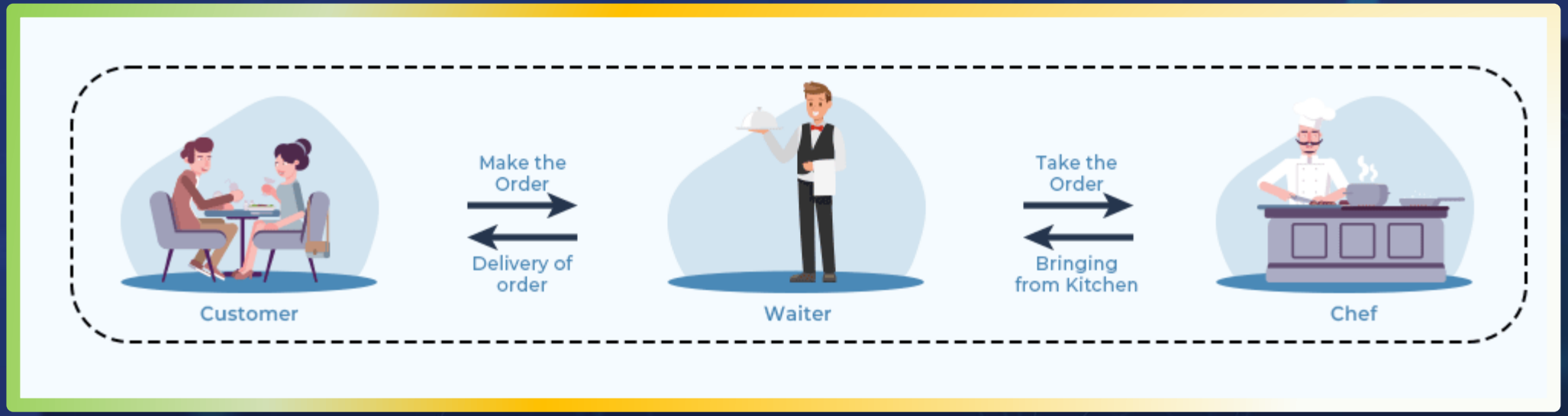
El protocolo HTTP (siglas de HyperText Transfer Protocol, o Protocolo de Transferencia de Hipertexto) es el lenguaje que utilizan los navegadores y los servidores web para comunicarse entre sí a través de internet.



Cuando abrís una página web, tu navegador (como Chrome o Firefox) envía una solicitud HTTP al servidor donde está alojado el sitio. El servidor responde con otra comunicación HTTP que contiene la información solicitada (por ejemplo, el texto, las imágenes o el código HTML de la página). Es como una conversación estructurada.

CENTRO DE GRADUADOS DE LA FACULTAD DE INGENIERÍA
TU EXITO PROFESIONAL

¿Que es una API (Application Programming Interface)?



Eres el Cliente (Usuario Final desde navegador) sentado en la mesa. Tienes un objetivo: comer. Miras un menú que te dice qué platos puedes pedir y cómo pedirlos. No sabes (ni te importa) cómo el Chef (El Servidor/Back-end) cocina la comida, qué ingredientes exactos usa o cómo funciona el horno. Solo quieres el plato final, que es tu "petición" (Request).

El Mesero (API) es el intermediario crucial. Toma tu pedido basado en el menú y lo anota en un formato (protocolo) que la cocina entienda. Una vez la cocina tiene tu “plato listo” (Response) te lo acerca a la mesa.

CENTRO DE GRADUADOS DE LA FACULTAD DE INGENIERÍA

TU EXITO PROFESIONAL

Tipo de datos comunes

- **Textos (string):** Palabras, fases entre comillas (ej: Nombre)
- **Numeros (number):** Valores numéricos enteros o decimales (ej: 100, 3,1416, -37)
- **Booleanos (boolean):** Verdadero o Falso (ej: activo = true)
- **Arreglo/Lista (array):** conjunto de valores en orden (ej: [1,2,23,10] o [“Omar”, “Juan”])
- **Objetos (object):** Conjunto de datos con nombre y valor (ej: {nombre: “Luis”, edad: 25})

JSON: pilar para datos

```json
{
  "id": 123,
  "username": "jperez",
  "nombre": "Juan",
  "apellido": "Pérez",
  "email": "juan.perez@example.com",
  "telefono": "+54 11 1234 5678",
  "rol": "admin",
  "activo": true,
  "creado_en": "2025-12-10T15:00:00Z",
  "direccion": {
    "calle": "Av. Siempre Viva 742",
    "ciudad": "Buenos Aires",
    "pais": "AR"
  }
}
```

JSON es un formato de texto para representar y enviar datos estructurados de forma sencilla entre sistemas. Se usa muchísimo en **desarrollo web y APIs** porque es fácil de leer por humanos y procesar por máquinas. Incluso los flujos de N8N son guardados en este formato.

El Motor del Backend: Python y FastAPI

Para construir nuestro backend elegimos FastAPI. En el mundo del software moderno y en particular en el ecosistema de Inteligencia Artificial, Python es el rey absoluto. Y FastAPI es el framework más moderno y veloz que existe en Python:



Diseñado en Python moderno; rendimiento a la par de Node.js o Go



(Pydantic): Si el Frontend envía una fecha mal formada o un campo vacío, FastAPI rechaza la petición con un error 400 antes de tocar la base de datos.



(Swagger UI) Genera un panel visual interactivo en la ruta /docs sin escribir una sola línea de documentación manual.

Persistencia Real: ¿Por qué SQLite?

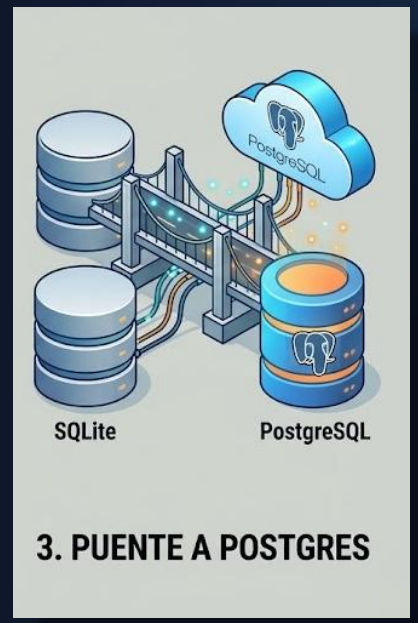
La base de datos relacional estándar sin fricción de configuración para desarrollo local.



(Zero-Config): Viene integrado de forma nativa en Python. No requiere instalar servidores, crear usuarios ni configurar contraseñas ni puertos.



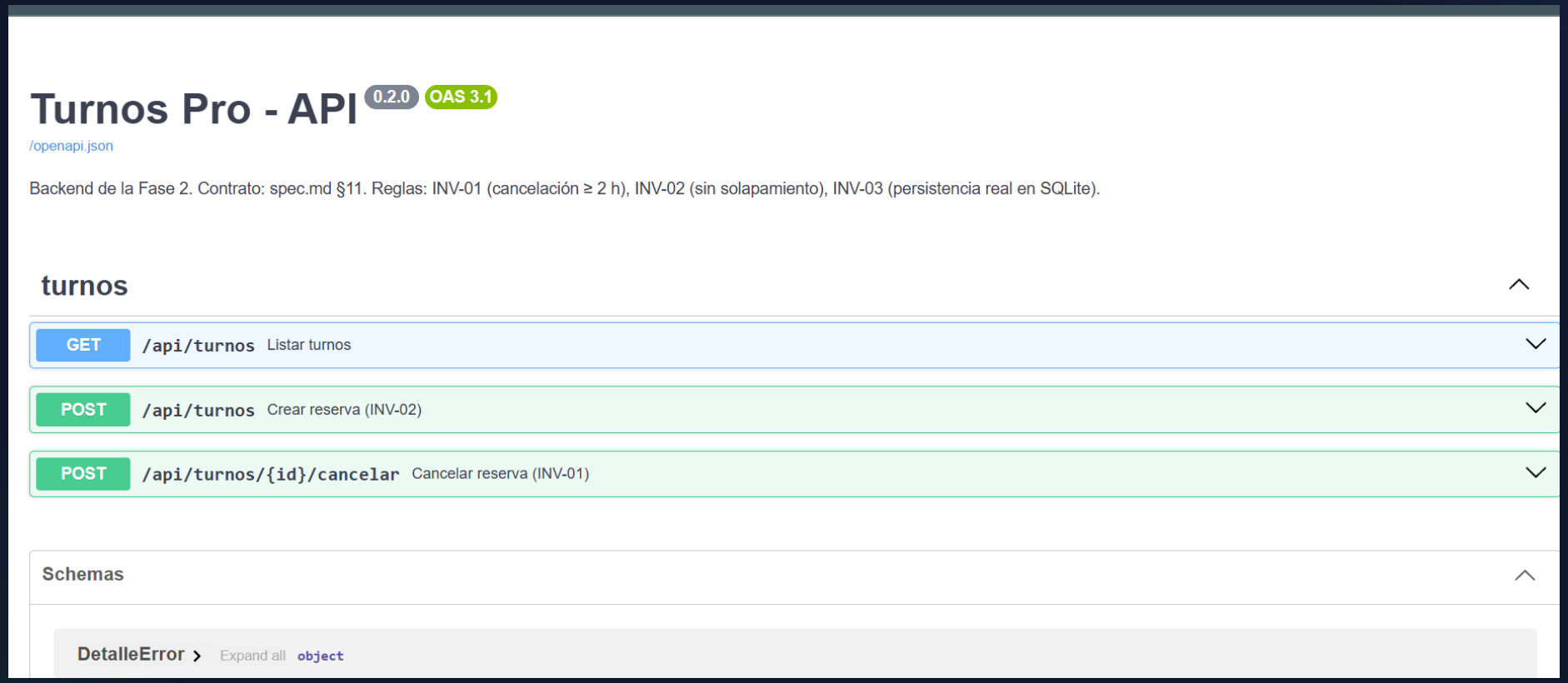
(ACID Real): Tablas estructuradas, tipos de datos, claves primarias y relaciones íntegras. No se corrompe ante escrituras concurrentes como un archivo de texto o JSON.



(Fácil Migración): Emplea la misma sintaxis y estándar SQL que PostgreSQL. Pasar a un servidor en la nube o producción será cambiar una sola línea de conexión.

La Ventana de Auditoría: Swagger UI (/docs)

No necesitamos adivinar qué hace el backend ni programar una interfaz para auditarlo; la API se prueba a sí misma en tiempo real



Hoja de Ruta del Demo en Vivo

Vamos a abrir Claude y ejecutar los 3 prompts en secuencia. Veremos como actualizar la arquitectura de nuestro proyecto partiendo de una actualización de la especificación.

- Prompt 1 actualizar SPEC:MD
- Prompt 2 Generar código
- Prompt 3 Refactorizar

HITO DE LA CLASE 4

Misión Cumplida: Mejora de Arquitectura

Los logros de hoy

- ✓ Repositorio clonado y sincronizado con Git.
- ✓ Servidor FastAPI modular corriendo en local.
- ✓ Persistencia de datos permanente sin pérdidas al recargar.
- ✓ Frontend y Backend comunicados mediante API REST.

Desafío semanal

Refinar las reglas de tu spec.md y hacer un segundo commit con tus avances.

CENTRO DE GRADUADOS DE LA FACULTAD DE INGENIERÍA

TU EXITO PROFESIONAL

PROXIMA SEMANA

Programá como un profesional.

¿Cómo podemos controlar que todo lo trabajado, continue funcionando?

TESTING

Testing automatizado con pytest: cómo validar 50 reglas en 2 segundos.

Refactorización asistida y blindaje de endpoints.



CENTRO DE GRADUADOS DE LA FACULTAD DE INGENIERÍA

TU EXITO PROFESIONAL

Muchas Gracias

POR SU ATENCIÓN.



